ROADMAP DATA ENGINEERING

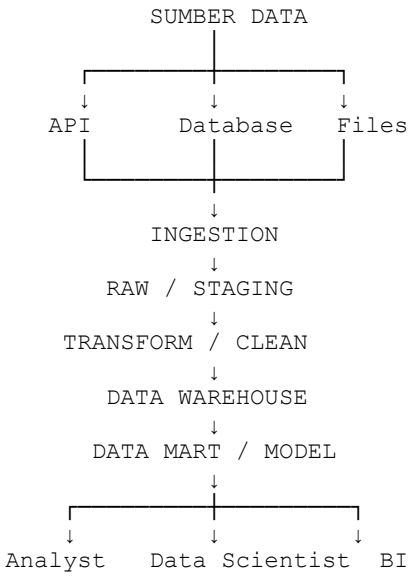
Beginner → Intermediate

Gua bikin sampai level yang menurut gua sudah cukup untuk lu mulai masuk ke **real-world/junior Data Engineering workflow**, bukan sampai expert.

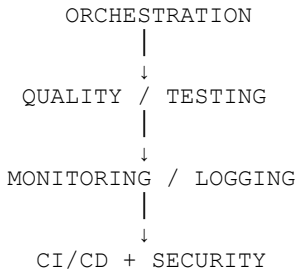
---

0. GAMBAR BESARNYA DULU

Sebelum masuk materi, lu harus punya mental model seperti ini:



Dan di production ada lapisan tambahan:



Itu gambaran pekerjaan Data Engineer secara umum: mengintegrasikan, mentransformasi, dan mengonsolidasikan data dari berbagai sumber lalu menyediakan data yang andal untuk kebutuhan analitik. (Apache Airflow)

🟢 LEVEL 1 — BEGINNER

Target akhir:

“Gua mampu membuat pipeline data batch yang rapi, teruji, bisa dijalankan otomatis, dan ngerti kenapa setiap komponennya dipakai.”

MODULE 1 — FUNDAMENTAL DATA ENGINEERING

Materi

Lu belajar dulu:

- apa itu Data Engineering
- data pipeline
- source
- ingestion
- raw data
- staging
- transformation
- destination
- ETL vs ELT
- batch vs streaming
- OLTP vs OLAP
- database vs warehouse
- data lake
- data warehouse
- data mart
- data pipeline failure
- idempotency
- incremental processing

Kasus nyata

Kasus:

Sebuah toko punya:

```
orders.csv
customers.csv
products.csv
```

Bos minta:

“Gua mau laporan penjualan bulanan yang datanya otomatis dibersihkan.”

Lu harus menentukan:

```
CSV
 ↓
Extract
 ↓
```

```
Clean
 ↓
Transform
 ↓
PostgreSQL
 ↓
Sales Report
```

Penilaian

Lu harus mampu menjawab:

Data mana yang source?

Mana raw?

Mana transformed?

Kenapa perlu staging?

Kenapa tidak langsung query CSV?

ETL atau ELT?

Batch atau streaming?

GATE

Lulus kalau lu bisa **menggambar pipeline dari sebuah kasus bisnis tanpa melihat catatan**.

MODULE 2 — PYTHON FOR DATA ENGINEERING

Lu nggak perlu belajar Python seperti software engineer dulu.

Fokus ke Python yang dipakai buat data pipeline.

Materi

- function
- module
- package
- exception
- file handling
- CSV
- JSON
- pathlib
- datetime
- environment variable
- logging

- requests
- pandas
- typing dasar
- class secukupnya
- virtual environment
- package management

Lalu mulai:

```
script.py
```

berubah menjadi:

```
project/
├── src/
│   ├── extract.py
│   ├── transform.py
│   ├── load.py
│   └── main.py
├── tests/
├── config/
└── requirements.txt
```

Kasus

API menyediakan:

```
{
  "customer_id": 101,
  "name": "Saifullah",
  "amount": "150000",
  "date": "2026-09-30"
}
```

Data kadang:

```
amount = "150000"
amount = null
amount = "Rp150.000"
```

Tugas lu:

bangun pipeline Python yang mengambil data → membersihkan → menghasilkan dataset bersih.

Penilaian

Lu dikasih API/data kacau.

Lu harus mampu:

```
extract()
transform()
load()
```

tanpa membuat semuanya dalam satu file `main.py`.

GATE

Kalau API mati, program **tidak boleh langsung crash tanpa informasi**.

Harus ada:

```
error message
logging
exception handling
```

MODULE 3 — SQL MENDALAM

Ini salah satu modul **paling penting** buat lu.

Bukan cuma:

```
SELECT *
FROM orders;
```

Tapi sampai lu mampu berpikir menggunakan SQL.

Materi

Urutan:

```
SELECT
WHERE
GROUP BY
HAVING
ORDER BY
JOIN
CASE
CTE
Subquery
Window Function
Aggregation
Date/Time
NULL handling
UNION
EXISTS
```

Kemudian:

```
INDEX
EXPLAIN
QUERY PLAN
TRANSACTION
VIEW
MATERIALIZED VIEW
```

Kasus

Database e-commerce:

```
customers
orders
order_items
products
```

Bos bertanya:

Siapa customer dengan total pembelian terbesar?

Produk paling laku?

Revenue bulan ini?

Customer yang belum belanja 90 hari?

Repeat customer?

Average order value?

Top 3 produk per kategori?

Di sini lu mulai bekerja seperti Data Engineer/Analytics Engineer, bukan sekadar belajar SQL.

GATE

Lu diberi database baru yang **belum pernah lu lihat**.

Lu harus mampu:

```
inspect schema
↓
pahami relationship
↓
buat query
↓
validasi hasil
```

tanpa mengandalkan tutorial langkah-per-langkah.

MODULE 4 — DATABASE FUNDAMENTAL

Karena lu sudah pakai PostgreSQL, sekarang kita naik level.

Materi

- primary key
- foreign key
- normalization
- constraints
- indexes
- transactions
- ACID
- isolation
- locks
- views
- stored procedure dasar

- database permissions
- connection
- pooling dasar

Kasus nyata

Bayangkan lu diminta membuat database untuk:

Sistem inventaris kampus.

Terdapat:

```
barang
pegawai
ruangan
supplier
barang_masuk
barang_keluar
detail_barang_keluar
```

Lu harus menentukan:

```
1 barang
↓
bisa punya banyak transaksi
```

bukan menduplikasi nama barang di mana-mana.

Ini bahkan sangat dekat dengan sistem Menventory yang pernah lu desain.

GATE

Lu diberi requirement:

“Buat database sistem gudang.”

Lu harus mampu membuat:

```
ERD
↓
schema
↓
constraints
↓
indexes
```

dan menjelaskan alasan desainnya.

MODULE 5 — DATA INGESTION

Sekarang mulai benar-benar belajar:

Bagaimana data masuk ke pipeline?

Materi

Sumber:

```
CSV
JSON
Excel
REST API
PostgreSQL
MySQL
```

Kemudian konsep:

```
full load
incremental load
watermark
upsert
duplicate
schema change
pagination
rate limit
retry
checkpoint
```

Kasus nyata

API transaksi mempunyai:

```
1.000.000 records
```

Tapi API hanya mengizinkan:

```
100 record/request
```

Lu harus membuat:

```
API
 ↓
pagination
 ↓
extract
 ↓
raw JSON
 ↓
database
```

Kemudian hari berikutnya:

jangan download 1 juta data lagi.

Ambil:

```
data baru saja
```

GATE

Lu harus mampu membangun:

```
incremental ingestion
```

dan menjelaskan:

“Data terakhir yang berhasil diproses sampai ID/timestamp berapa?”

MODULE 6 — ETL / ELT ENGINEERING

Nah, ini wilayah yang sudah pernah lu sentuh.

Sekarang kita bikin jauh lebih proper.

Materi

```
Extract
Transform
Load
```

dan:

```
ELT
Staging
Raw layer
Clean layer
Mart
Incremental transformation
Idempotency
Retry
Logging
```

Kasus

Data:

```
orders
customers
products
```

masuk setiap hari.

Pipeline:

```
API
 ↓
raw_orders
 ↓
clean_orders
 ↓
orders_fact
 ↓
daily_sales
```

Lu harus menangani:

```
duplicate
```

```
missing value
invalid date
wrong datatype
negative quantity
unknown customer
```

GATE

Pipeline dijalankan 2×.

Hasilnya **tidak boleh menggandakan data**.

Ini penting banget.

Karena pipeline production tidak cukup:

“sekali jalan berhasil.”

Tapi harus:

“kalau retry, hasil tetap benar.”

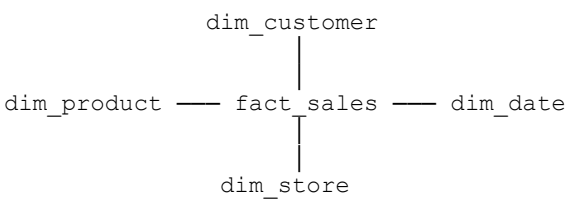
MODULE 7 — DATA MODELING

Sekarang lu belajar bagaimana **data disusun untuk analytics**.

Materi

- fact table
- dimension table
- star schema
- snowflake schema
- grain
- surrogate key
- slowly changing dimension
- historical data
- data mart

Contoh



Kemudian:

```
fact_sales

sale_id
```

```
customer_key
product_key
store_key
date_key
quantity
revenue
```

Kasus

Management minta:

“Dashboard penjualan berdasarkan customer, produk, toko, dan tanggal.”

Lu harus mendesain warehouse-nya.

GATE

Lu harus bisa menjawab:

Apa grain fact table ini?

Kalau lu belum bisa menjawab pertanyaan itu, **belum boleh lanjut**.

MODULE 8 — DATA QUALITY + TESTING

Ini yang sering dilewatkan pemula.

Padahal Data Engineer bukan cuma:

“data berhasil masuk.”

Tapi:

“data benar.”

Data Engineer memang punya tanggung jawab pada reliability, quality, dan pipeline yang dapat dipercaya. (Apache Airflow)

Materi

Validasi:

```
NOT NULL
UNIQUE
PRIMARY KEY
FOREIGN KEY
accepted values
range
row count
freshness
duplicate
referential integrity
```

Kasus

Pipeline menghasilkan:

```
1.000.000 rows
```

kemarin.

Hari ini:

```
12.000 rows
```

Pipeline tetap sukses.

Tapi sebenarnya:

API upstream rusak.

Lu harus mampu mendeteksinya.

GATE

Lu diberi pipeline yang sengaja dibuat rusak.

Lu harus menemukan:

```
apa yang rusak?
di tahap mana?
kenapa?
bagaimana mencegahnya?
```

MODULE 9 — GIT + LINUX + CLI

Ini masuk **workflow engineering**.

Materi

Git:

```
init
clone
branch
commit
merge
pull
push
rebase dasar
.gitignore
```

Linux:

```
ls
cd
cp
```

```
mv
rm
cat
grep
find
chmod
ps
top
curl
wget
```

Environment:

```
.env
environment variables
virtual environment
```

Kasus

Lu dikasih server Linux.

Pipeline Python lu harus dijalankan di sana.

GATE

Lu tidak boleh bergantung pada:

“klik tombol Run di VS Code.”

Lu harus mampu:

```
python -m ...
```

dari terminal.

MODULE 10 — DOCKER

Baru sekarang masuk Docker.

Jangan terlalu awal.

Karena Docker sebelum ngerti aplikasi lu cuma bikin:

“gua tau command Docker tapi nggak ngerti kenapa.” 🤦

Materi

- image
- container
- Dockerfile
- volume
- network

- environment
- Docker Compose

Kasus

Lu punya:

```
Python ETL
PostgreSQL
```

Buat supaya seluruh project bisa dijalankan:

```
docker compose up
```

GATE

Teman lu clone GitHub project lu.

Dia cukup menjalankan:

```
docker compose up
```

dan pipeline bisa jalan.

MODULE 11 — AIRFLOW

Ini milestone besar pertama lu.

Airflow digunakan untuk mendefinisikan, menjadwalkan, dan memantau workflow berbentuk DAG dengan task dan dependency. (Apache Airflow)

Materi

```
DAG
Task
Operator
Dependency
Schedule
Retry
Timeout
Backfill
Variables
Connections
Logs
Task failure
```

Kasus nyata

Pipeline:

```
extract API
      ↓
validate
```

```
      ↓
transform
      ↓
load PostgreSQL
      ↓
run SQL aggregation
```

Di Airflow:

```
extract
   ↓
validate
   ↓
transform
   ↓
load
   ↓
aggregate
```

Skenario failure

Lu sengaja matikan API.

Airflow harus:

```
extract ❌
```

kemudian:

```
retry
retry
retry
```

dan tidak menjalankan task berikutnya sebelum dependency terpenuhi.

GATE

Lu mampu membuat DAG sendiri dari requirement:

“Setiap hari jam 02:00, ambil transaksi kemarin, transform, load, lalu validasi.”

Tanpa tutorial.

🟡 SELESAI BEGINNER

Di titik ini, stack utama lu kira-kira:

```
Python
SQL
PostgreSQL
Git
Linux
REST API
ETL/ELT
Data Modeling
```

```
Testing
Docker
Airflow
```

Dan **ini sudah jauh lebih berharga** daripada hafal 20 tools tetapi tidak bisa bikin pipeline.

🟠 LEVEL 2 — INTERMEDIATE

Sekarang baru kita masuk dunia data platform yang lebih besar.

MODULE 12 — DATA WAREHOUSE

Belajar:

```
OLTP
     ↓
ETL/ELT
     ↓
Data Warehouse
```

Contoh:

```
BigQuery
Snowflake
Redshift
```

Tidak perlu semuanya.

Pilih satu.

Untuk belajar konsep, lu bahkan bisa mulai lokal menggunakan warehouse-style engine seperti DuckDB sebelum masuk cloud.

Materi

- OLAP
- columnar storage
- partitioning
- clustering
- warehouse modeling
- analytical query
- cost awareness

Kasus

Data:

```
100 juta transaksi
```

Query laporan harus tetap efisien.

Lu harus belajar:

kenapa query tertentu lambat?

kapan partition berguna?

kenapa data warehouse berbeda dengan PostgreSQL OLTP?

MODULE 13 — dbt

Ini mulai masuk workflow modern transformation.

dbt memungkinkan transformation SQL dibuat modular, version-controlled, terdokumentasi, dan diberi data tests; dbt juga bisa mendokumentasikan lineage/model metadata. (dbt Developer Hub)

Materi

```
models
sources
staging
marts
ref()
tests
documentation
lineage
incremental models
```

Kasus

Data:

```
raw_orders
raw_customers
raw_products
```

Lu bikin:

```
stg_orders
stg_customers
stg_products

        ↓

fct_orders
dim_customers
dim_products

        ↓

mart_sales
```

GATE

Lu diberi raw dataset.

Lu harus mampu membuat:

```
source
↓
staging
↓
dimension/fact
↓
tests
↓
documentation
```

tanpa semuanya dilakukan dengan Python.

MODULE 14 — CLOUD FUNDAMENTAL

Jangan belajar AWS 100 layanan.

Belajar konsep dulu.

Misalnya AWS:

```
S3
IAM
EC2
RDS
Glue
Athena
Redshift
```

Tapi lu **nggak perlu mendalami semuanya sekaligus**.

Fokus:

```
storage
compute
database
network
IAM
cost
```

Kasus

Arsitektur:

```
API
 ↓
S3
 ↓
processing
 ↓
warehouse
 ↓
BI
```

Lu harus memahami:

kenapa data mentah masuk object storage?

kenapa tidak semuanya di PostgreSQL?

kapan pakai warehouse?

MODULE 15 — SPARK / PYSPARK

Baru sekarang masuk Spark.

Spark digunakan untuk workload data skala besar dan mendukung batch serta streaming. (Apache Airflow)

Materi

- distributed processing
- Spark architecture
- DataFrame
- Spark SQL
- transformations
- actions
- partition
- shuffle
- join
- caching
- lazy evaluation
- PySpark

Kasus

Data:

```
10 GB
100 GB
1 TB
```

Lu punya transformasi:

```
read
filter
join
groupBy
aggregate
write
```

Kemudian bandingkan:

```
pandas
vs
PySpark
```

GATE

Lu harus mampu menjelaskan:

Kenapa pandas mulai bermasalah?

Apa itu partition?

Apa yang menyebabkan shuffle?

Kenapa join bisa mahal?

MODULE 16 — STREAMING + KAFKA

Ini **ujung intermediate**, bukan beginner.

Kafka digunakan untuk event streaming ber-throughput tinggi dan pipeline yang fault-tolerant. (Apache Airflow)

Materi

```
event
producer
consumer
topic
partition
offset
consumer group
broker
retention
ordering
delivery semantics
```

Kasus

Website menghasilkan event:

```
{
  "user_id": 123,
  "event": "checkout",
  "timestamp": "..."
}
```

Setiap detik ada ribuan event.

Pipeline:

```
Website
   ↓
Kafka
   ↓
Consumer
   ↓
Processing
```

```
   ↓
Storage
   ↓
Dashboard
```

GATE

Lu harus memahami perbedaan:

```
Batch

vs

Streaming
```

dan bisa menjelaskan:

“Kenapa kasus ini membutuhkan streaming?”

Bukan sekadar:

“karena Kafka keren.” 😭

MODULE 17 — PRODUCTION ENGINEERING

Sekarang semua yang lu pelajari mulai digabung.

Materi

```
logging
monitoring
alerting
retry
timeout
dead letter
idempotency
backfill
recovery
data freshness
pipeline SLA
```

Kasus

Jam 03:00 pipeline gagal.

Lu harus bisa mencari:

```
task mana gagal?
↓
error apa?
↓
source atau transform?
↓
data hilang atau tidak?
↓
```

```
boleh retry?
↓
perlu backfill?
```

Ini mulai terasa seperti pekerjaan sungguhan.

MODULE 18 — CI/CD

Materi

```
GitHub Actions
testing
linting
deployment
environment
dev/staging/prod
```

Misalnya:

```
git push
   ↓
GitHub Actions
   ↓
run tests
   ↓
build
   ↓
deploy
```

MODULE 19 — SECURITY + GOVERNANCE

Tidak perlu langsung jadi security engineer.

Pahami:

```
authentication
authorization
RBAC
IAM
secrets
encryption
PII
data masking
audit
```

Kasus

Tabel:

```
customers
```

berisi:

```
name
```

```
email
phone
address
```

Business analyst tidak seharusnya otomatis mendapatkan seluruh data sensitif.

Lu harus berpikir:

siapa boleh lihat apa?

MODULE 20 — CAPSTONE

Ini **ujian akhir Intermediate**.

Lu bikin satu project yang menggabungkan hampir semuanya.

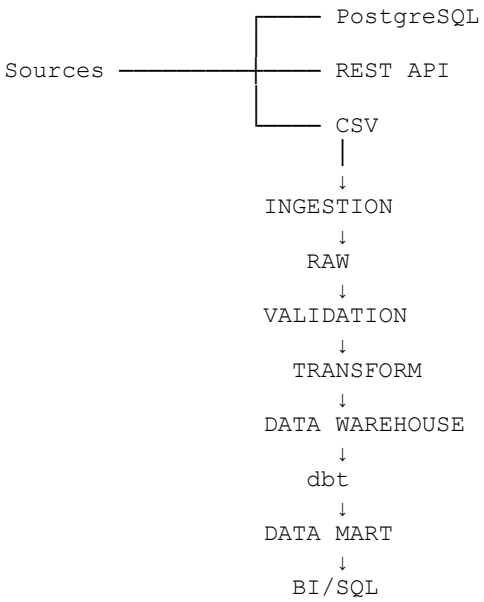
CASE:

E-Commerce Data Platform

Sumber:

```
PostgreSQL
REST API
CSV
```

Pipeline:



Orchestration:

```
Airflow
```

Infrastructure:

```
Docker
```

Versioning:

```
Git/GitHub
```

Testing:

```
dbt tests
Python tests
```

Monitoring:

```
logs
pipeline status
freshness
```

🔥 METODE BELAJAR LU

Nah ini bagian paling penting.

Setiap module jangan cuma:

baca → ngerti → lanjut.

Pakai siklus ini:

```
1. MATERI
      ↓
2. LATIHAN KECIL
      ↓
3. KASUS NYATA
      ↓
4. PENILAIAN
      ↓
5. EVALUASI
      ↓
6. REMEDIAL / ULANG
      ↓
7. GATE
      ↓
NEXT MODULE
```

FORMAT BELAJAR YANG GUA SARANKAN BUAT LU

Setiap kali kita mulai sebuah materi, formatnya bisa seperti ini:

MATERI

Gua jelasin:

```
konsep
↓
kenapa dibutuhkan
↓
cara kerja
↓
contoh
↓
hubungan dengan Data Engineering
```

Bukan langsung disuruh coding.

LATIHAN

Mulai dari:

```
basic
↓
moderate
↓
real case
```

CASE

Misalnya:

“Sebuah perusahaan memiliki 500.000 transaksi per hari…”

Lu diberi requirement.

Dan lu harus berpikir sendiri.

PENILAIAN

Gua cek:

```
Correctness
Logic
Data handling
Code quality
SQL quality
Error handling
Pipeline design
Understanding
```

Bukan sekadar:

hasil akhirnya benar/salah.

❗ GATE SYSTEM

Menurut gua ini bakal sangat cocok dengan gaya belajar lu.

Setiap module punya tiga kondisi:

🔴 BELUM KUASAI

Masih banyak konsep yang salah.

→ ulang materi + latihan.

🟡 PAHAM

Bisa mengerjakan tapi masih banyak dibantu.

→ tambah kasus.

🟢 KUASAI

Bisa:

```
menjelaskan
+
mengerjakan
+
debug
+
mengambil keputusan
```

→ baru lanjut.

🧠 DAN JANGAN BELAJAR SEMUA TOOL

Ini penting banget.

Roadmap lu **jangan** jadi:

```
Python
Java
Scala
Kafka
Flink
Spark
Hadoop
NiFi
Airbyte
Fivetran
Snowflake
```

```
BigQuery
Redshift
Azure
AWS
GCP
Kubernetes
Terraform
...
```

💀

Untuk Beginner → Intermediate, gua lebih milih **depth > quantity**.

Stack utama lu:

```
Python
SQL
PostgreSQL
Git
Linux
REST API
ETL/ELT
Docker
Airflow
dbt
Data Warehouse
Cloud basics
PySpark
Kafka
```

Itu sudah jadi jalur yang sangat solid.

🗺️ URUTAN FINAL YANG GUA REKOMENDASIKAN

```
BEGINNER
│
├── 01 Data Engineering Fundamentals
├── 02 Python for Data Engineering
├── 03 Advanced SQL
├── 04 Database Engineering
├── 05 Data Ingestion
├── 06 ETL / ELT
├── 07 Data Modeling
├── 08 Data Quality & Testing
├── 09 Git + Linux
├── 10 Docker
└── 11 Airflow
        │
        ▼
   JUNIOR-READY BASE
        │
        ▼
INTERMEDIATE
│
├── 12 Data Warehouse
├── 13 dbt
├── 14 Cloud Fundamentals
├── 15 PySpark
```

```
├── 16 Kafka / Streaming
├── 17 Monitoring & Reliability
├── 18 CI/CD
├── 19 Security & Governance
└── 20 Capstone
```

Yang paling penting:

Lu **nggak perlu menunggu sampai module 20 untuk punya portfolio**.

Justru setiap beberapa module kita bisa menghasilkan project:

```
Project 1
Python ETL

Project 2
API → PostgreSQL

Project 3
Incremental ETL

Project 4
Airflow ETL

Project 5
Dockerized Data Pipeline

Project 6
dbt Warehouse

Project 7
Spark Pipeline

Project 8
Kafka Streaming

Project 9
Full Data Platform
```

Jadi GitHub lu berkembang **seiring kemampuan**, bukan baru bikin portfolio setelah belajar semuanya.

Dan gua sengaja menaruh **Airflow setelah fondasi pipeline** karena Airflow memang berfungsi mengorkestrasi workflow—menentukan task, dependency, schedule, retry, monitoring—bukan menggantikan logic ETL itu sendiri. (Apache Airflow)

Begitu juga **dbt baru masuk setelah lu kuat SQL + modeling**, karena kekuatannya ada pada transformasi SQL yang modular, testing, documentation, dan lineage. (dbt Developer Hub)

Jadi roadmap kita mulai bukan dari "belajar tool", tapi dari "belajar menjadi Data Engineer".